Construction Intelligence Platform — Target Architecture

Oct 12, 2026 · @Sakshi

1. Decisions

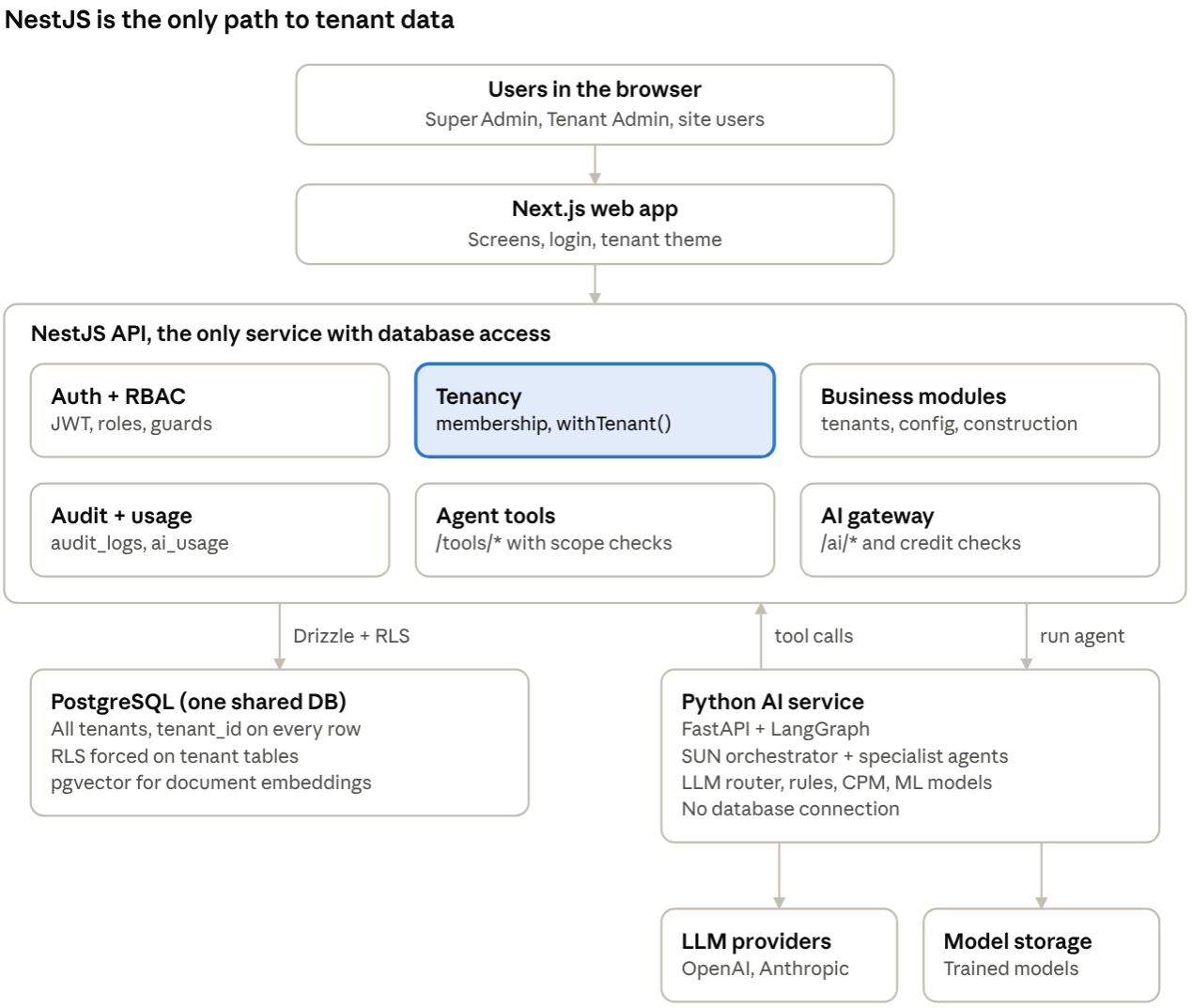
All tenants share one application and one PostgreSQL database, isolated by `tenant_id` and Row Level Security. NestJS is the only service that touches tenant data; the Python AI service reaches it through NestJS tool APIs.

Area	Decision	Why
Tenancy model	One shared PostgreSQL DB, `tenant_id` on every tenant-owned table, RLS enforced	One migration path, one connection pool, one agent tool gateway
Data ownership	NestJS + Drizzle own all reads and writes	One security system: auth, RBAC, tenant context, audit
AI service	Python + FastAPI + LangGraph, stateless, no tenant DB connection	Python ecosystem for LLM/ML; isolation stays in NestJS
Agent data access	Agents call NestJS tool endpoints with a signed internal token (tenant + scopes)	Agents cannot reach another tenant's rows
Vector search	pgvector in the same DB with RLS (Pinecone namespaces optional later)	Same isolation rule covers embeddings
ML models	Shared base model + tenant-specific model once a tenant has 500+ records	Matches the AI standard; registry keyed by tenant
AI cost	Every LLM, embedding and search call writes an `ai_usage` row; plans sell AI Credits	Cost per tenant is known and limitable
Escape hatch	`tenants.data_location` (default `shared`) behind one `TenantDbService`	A large or regulated client can move to a dedicated DB without code changes

Database per tenant (the PropTech model) is not used. It multiplies migrations, connection pools and agent routing by the number of tenants, and blocks shared model training.

2. System architecture

The request path is a straight line of trust: the browser talks only to Next.js, Next.js only to NestJS, and only NestJS talks to PostgreSQL.

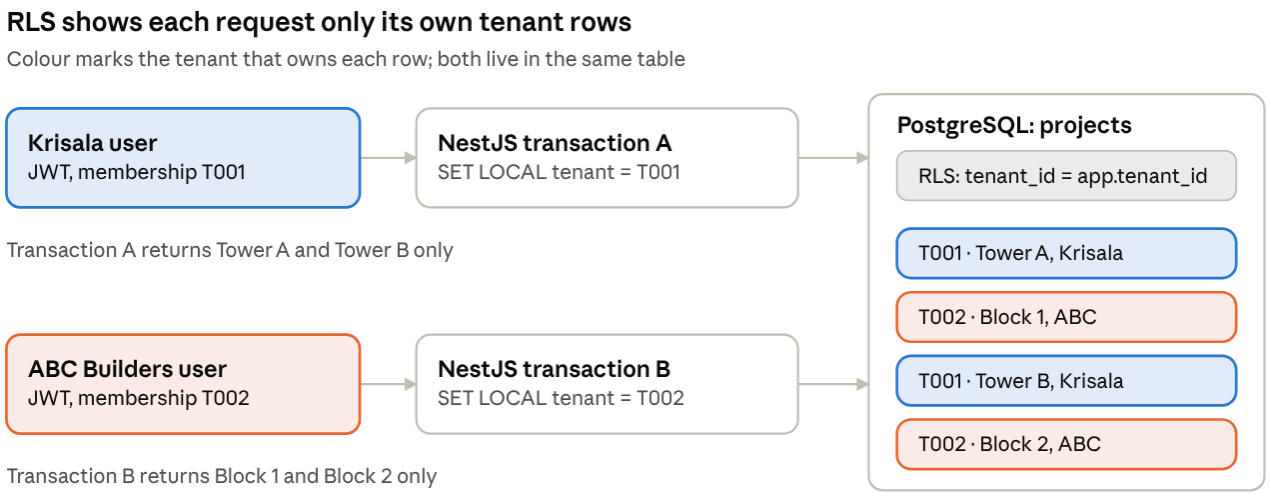


System architecture · web, NestJS, PostgreSQL, Python AI service

The Python AI service is called by NestJS and calls back into NestJS tool endpoints for data; it holds no database credentials. Redis (cache, per-tenant rate limits) and object storage for uploaded files sit beside NestJS and are omitted here for clarity.

3. Multi-tenant isolation model

Every store is shared, and every store is partitioned by `tenant_id` that only NestJS sets.



Tenant isolation · two tenants, one table, RLS filter

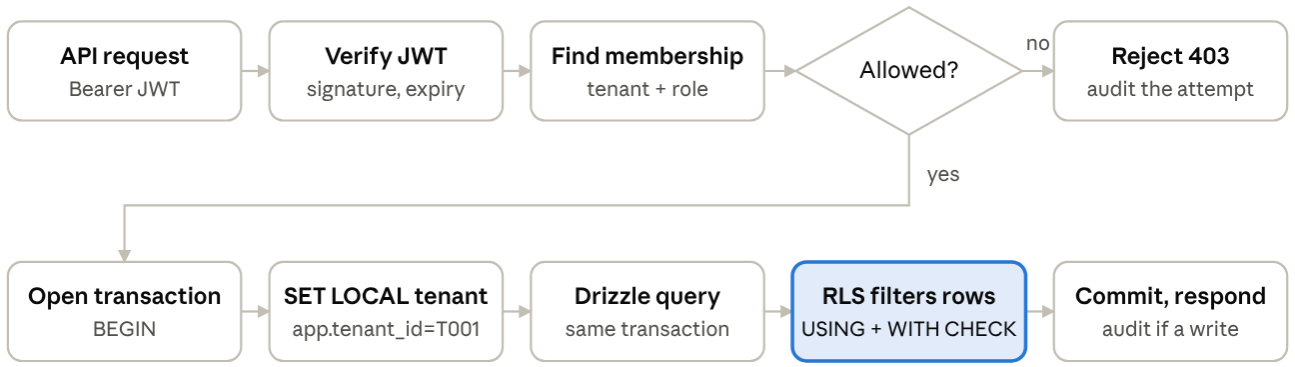
Even a query that forgets `WHERE tenant_id = ...` returns only the current tenant's rows, because PostgreSQL applies the policy itself.

Store	Shared?	How a tenant is isolated
PostgreSQL tables	One DB	`tenant_id` column + RLS policy using `app.tenant_id`
Embeddings (pgvector)	Same DB	`document_chunks.tenant_id` + the same RLS policy
Files (drawings, photos, PDFs)	One bucket	Key prefix `tenants/<tenant_id>/...`; signed URLs issued by NestJS
ML models	One registry	Key `tenant_<id>/<model>` or `base/<model>`
Redis cache	One instance	Key prefix `t:<tenant_id>:`
LLM calls	Shared providers	One tenant's data per call; usage metered per tenant
Logs and traces	Shared	`tenant_id` on every entry

A tenant with a contractual need for physical separation gets `data_location = dedicated`; `TenantDbService` then opens that tenant's own database with the same schema and RLS.

4. Flow 1 — Normal tenant request

Every tenant-scoped request runs inside one transaction that sets the tenant first, so the query and the RLS check share one connection.



Flow 1 · tenant request through NestJS and RLS

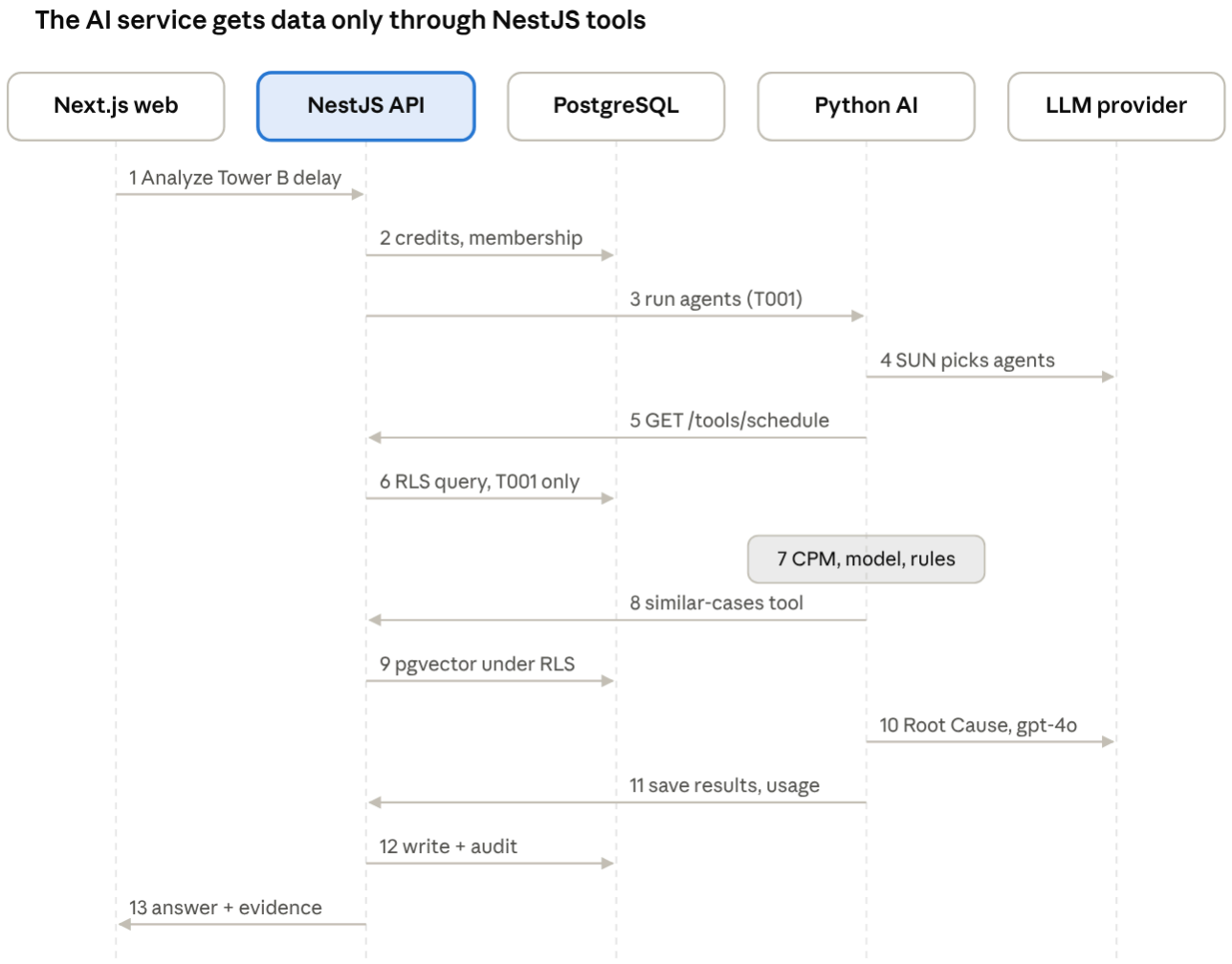
The NestJS helper that implements this, used by every service:

```
async withTenant<T>(tenantId: string, fn: (tx: Tx) => Promise<T>): Promise<T>
{
  return this.db.transaction(async (tx) => {
    await tx.execute(sql`select set_config('app.tenant_id', ${tenantId},
true)`);
    return fn(tx);
  });
}
```

`set_config(..., true)` is the parameterised form of `SET LOCAL`; the setting ends with the transaction, so a pooled connection never carries one tenant into the next request.

5. Flow 2 — AI agent request

Example: a Krisala site engineer clicks Analyze delay on Tower B. NestJS fixes the tenant at step 2, and every data step after that goes back through NestJS under RLS.



Flow 2 · Analyze delay on Tower B for Krisala, 13 steps

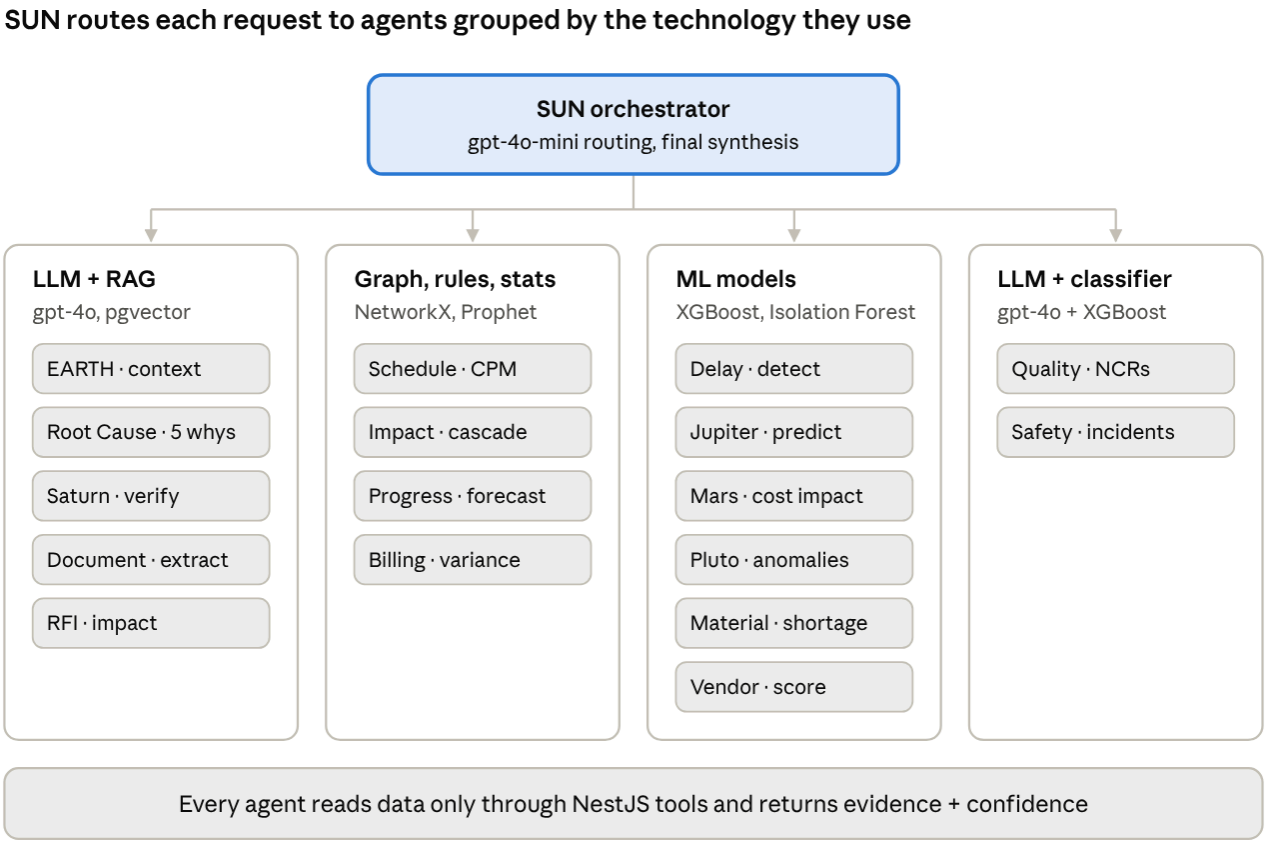
Step	What happens	Security check
1–2	NestJS verifies the JWT, resolves membership T001, checks remaining AI Credits	Tenant decided here, once
3	NestJS calls the Python service with a signed token: tenant T001, user, request id, scopes `schedule:read`, `delay:read`, `docs:search`, short expiry	Python cannot widen scope or change tenant
4	SUN (gpt-4o-mini) classifies intent and plans Schedule → Delay → Root Cause	Prompt contains only T001 data
5–6	Schedule agent calls `/tools/schedule`; NestJS runs `withTenant(T001)`	Token tenant re-checked; RLS on the query
7	Python runs CPM (NetworkX), the delay model (tenant or base) and rules	No data leaves the request

Step	What happens	Security check
8–9	EARTH calls `/tools/similar-cases`; pgvector search under RLS	Only T001 embeddings searchable
10	Root Cause (gpt-4o) reasons over the evidence; Saturn scores confidence	Output must cite evidence ids
11–12	Python posts findings and usage; NestJS writes `ai_findings`, `agent_runs`, `ai_usage`, `audit_logs`	Writes pass RLS `WITH CHECK`
13	The UI shows the answer with evidence and confidence	Same membership as step 1

Long analyses run asynchronously: step 3 returns a job id, and the UI receives the result by polling or a websocket.

6. Agent topology

SUN is the single entry point for every AI request; the 17 specialist agents are stateless functions that SUN calls, grouped by the technology the AI standard assigns them.



Agent topology · SUN plus 17 specialist agents in four technology groups

Every agent uses one input and output contract:

```
AgentInput  { tenant_id, request_id, project_id?, question, scopes[] }
AgentOutput { agent, result, evidence[{type, id}], confidence 0-1,
credits_used }
```

Agents keep no state between calls; run state lives in `agent_runs` (via NestJS) and Redis. A timeout or failure in one agent returns a partial answer from SUN instead of failing the request.

7. Flow 3 — Tenant onboarding

A new tenant is a set of rows written in one transaction; no database, schema or deployment is created.

1. Super Admin logs in; NestJS issues a JWT with platform role `SUPER_ADMIN`.
2. Super Admin submits name, code and status for the new tenant (for example ABC Builders, code `ABC`).
3. NestJS opens one transaction and writes:
   a. `tenants` row (`data_location = shared`, `allow_shared_training` per contract)
   b. `tenant_configs` row with default branding, theme, locale and feature flags
   c. `users` row for the Tenant Admin (Argon2 password hash)
   d. `tenant_memberships` row linking the user to the tenant as `TENANT_ADMIN`
   e. `tenant_ai_plans` row with the plan's monthly AI Credits (from Phase 3)
   f. `audit_logs` row: who created which tenant and admin
4. Commit. Any failure rolls everything back, so no half-created tenant exists.
5. The Tenant Admin logs in; NestJS resolves the membership and loads that tenant's theme.

From Phase 3, a background job also prepares the tenant's vector scope and assigns the base ML models until the tenant has enough data for its own.

8. Flow 4 — AI usage metering and credits

No AI call runs without a tenant, and every call that costs money writes one `ai_usage` row against that tenant.

1. **Before the call:** NestJS checks the tenant's remaining AI Credits and per-tenant rate limit. At 100% it blocks or warns, per plan.
2. **During the call:** the Python service picks the cheapest model that fits (gpt-4o-mini for routing and simple text, gpt-4o for Root Cause and Saturn).

3. **After the call:** the Python service posts usage to NestJS `POST /tools/ai-usage`, which writes the row under RLS.
4. **Daily job:** sums usage per tenant, sends alerts at 80%, and flags overage for billing.

Field	Meaning
`tenant_id`	Which customer
`user_id`	Who triggered it (null for scheduled agents)
`agent_name`	e.g. `root_cause`
`model_name`	e.g. `gpt-4o-mini`
`tokens_in`, `tokens_out`	LLM size of the call
`credits_used`	Deducted from the plan
`estimated_cost_usd`	What the platform paid the vendor
`project_id`	Optional job site
`request_id`	Links all calls of one user action
`created_at`	When

Rules, CPM, SPI/CPI and XGBoost predictions are logged with `credits_used = 0`, so reports stay complete. LangFuse traces prompts for debugging; the `ai_usage` table stays the billing source of truth.

9. Flow 5 — ML training and model registry

Training is the one place that reads across tenants, so it runs as a separate batch job with its own audited role, never inside the live agents.

1. **Extract:** the training job connects with `ml_training_reader`, a read-only role on a read replica. It reads only tenants where `allow_shared_training = true`, plus each tenant's own data for its own model.
2. **Features:** `ml/features.py` builds the same features used at prediction time (activity_type, planned_duration, vendor history and so on). Tenant names and IDs are dropped from the shared base set.
3. **Train:** base model on the pooled data; a tenant model when that tenant has 500 or more records.
4. **Evaluate:** a new version is promoted only if it beats the current one on a held-out set.

5. **Register:** the artifact is stored in object storage and listed in `ml_models` with key, version, metrics and record count.
6. **Serve:** at prediction time the Python service picks the model:

```
def get_model(tenant_id, model_type):
    m = registry.get(f"tenant_{tenant_id}/{model_type}", status="active")
    if m and m.record_count >= 500:
        return m
    return registry.get(f"base/{model_type}", status="active")
```

Model	Algorithm	Used by	Earliest phase
Delay prediction	XGBoost	Delay, Jupiter	4
Cost impact	XGBoost	Mars	4
Risk scoring	XGBoost classifier	Pluto, Jupiter	4
Progress forecast	Prophet	Progress	4
Vendor performance	XGBoost	Vendor	5
Anomaly detection	Isolation Forest	Pluto	5

Until real data exists, the same agents run on rules and statistics, as the AI standard recommends.

10. Database structure by phase

Every table except the platform registries carries `tenant_id UUID NOT NULL`, has RLS enabled and forced, and has a `tenant_id` index.

Table	Phase	Tenant-owned (RLS)	Purpose
`tenants`	1	No (platform)	Registry: name, code, status, `data_location`, `allow_shared_training`
`users`	1	No (platform)	Identities, Argon2 hash
`roles`	1	No (platform)	`SUPER_ADMIN`, `TENANT_ADMIN`, later project roles
`tenant_memberships`	1	Yes	User → tenant → role

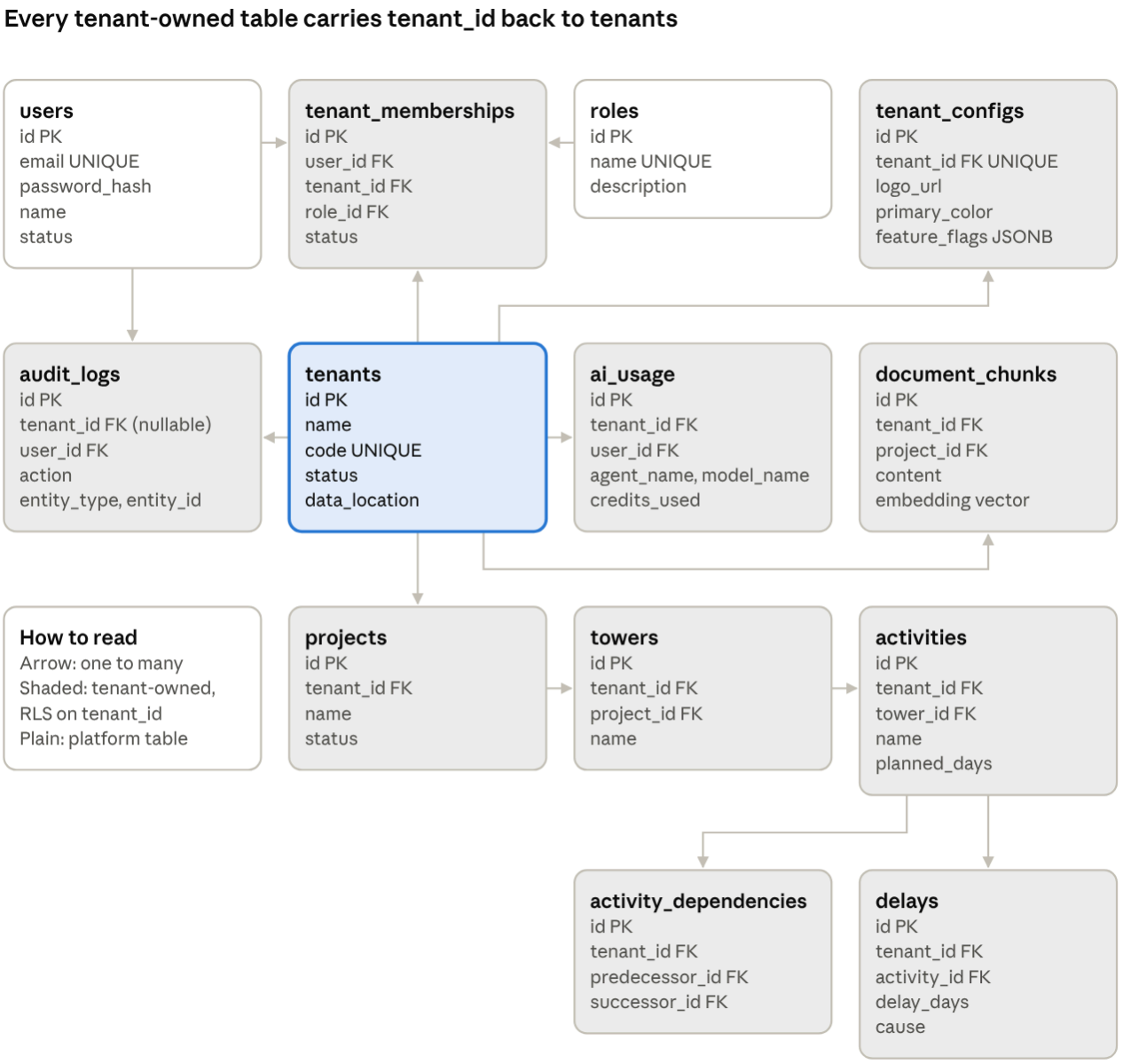
Table	Phase	Tenant-owned (RLS)	Purpose
`tenant_configs`	1	Yes	Branding, theme, locale, feature flags
`audit_logs`	1	Yes (tenant_id nullable for platform actions)	Admin, security and agent actions
`projects`, `sites`, `towers`, `floors`	2	Yes	Physical structure
`activities`, `activity_dependencies`	2	Yes	WBS and schedule graph for CPM
`progress_updates`, `delays`	2	Yes	Daily progress and delay events
`materials`, `material_stock`, `vendors`, `deliveries`	2	Yes	Supply side
`quality_ncrs`, `safety_incidents`, `rfis`, `documents`	2	Yes	Records the agents analyse
`tenant_ai_plans`, `ai_usage`	3	Yes	Credits and metering
`agent_runs`, `agent_steps`	3	Yes	Trace of each request: agent, input, output, evidence, confidence
`ai_findings`	3	Yes	Alerts and recommendations shown on dashboards
`document_chunks` (pgvector)	3	Yes	Embeddings for EARTH, Root Cause and Document agents
`ml_models`	4	No (registry, keyed by tenant or `base`)	Model versions, metrics, record counts
`ml_predictions`	4	Yes	Stored predictions for audit and retraining

Database roles

Role	Used by	Rights
`app_owner`	Migrations only	DDL, owns tables
`app_runtime`	NestJS	DML, no `BYPASSRLS`, RLS forced
`platform_admin_rw`	NestJS Super Admin endpoints	Platform tables; tenant tables only through explicit audited paths
`ml_training_reader`	Training batch job	Read-only on replica, audited
`readonly_inspect`	Beekeeper Studio for developers	Read-only; production access restricted

10.1 Table relationships (ERD)

`tenants` is the hub: every shaded table has a `tenant_id` foreign key to it, and RLS filters on that column.



Entity relationships, core tables for Phases 1 to 3

The chain under a tenant is projects, towers, activities, delays. `towers`, `activities` and `delays` also carry `tenant_id` directly, not only through their parent, so RLS checks one column with no joins. A composite foreign key keeps each child inside its parent's tenant:

```
CREATE TABLE towers (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  project_id  uuid NOT NULL,
  name        text NOT NULL,
  FOREIGN KEY (tenant_id, project_id) REFERENCES projects (tenant_id, id)
);
-- projects needs: UNIQUE (tenant_id, id)

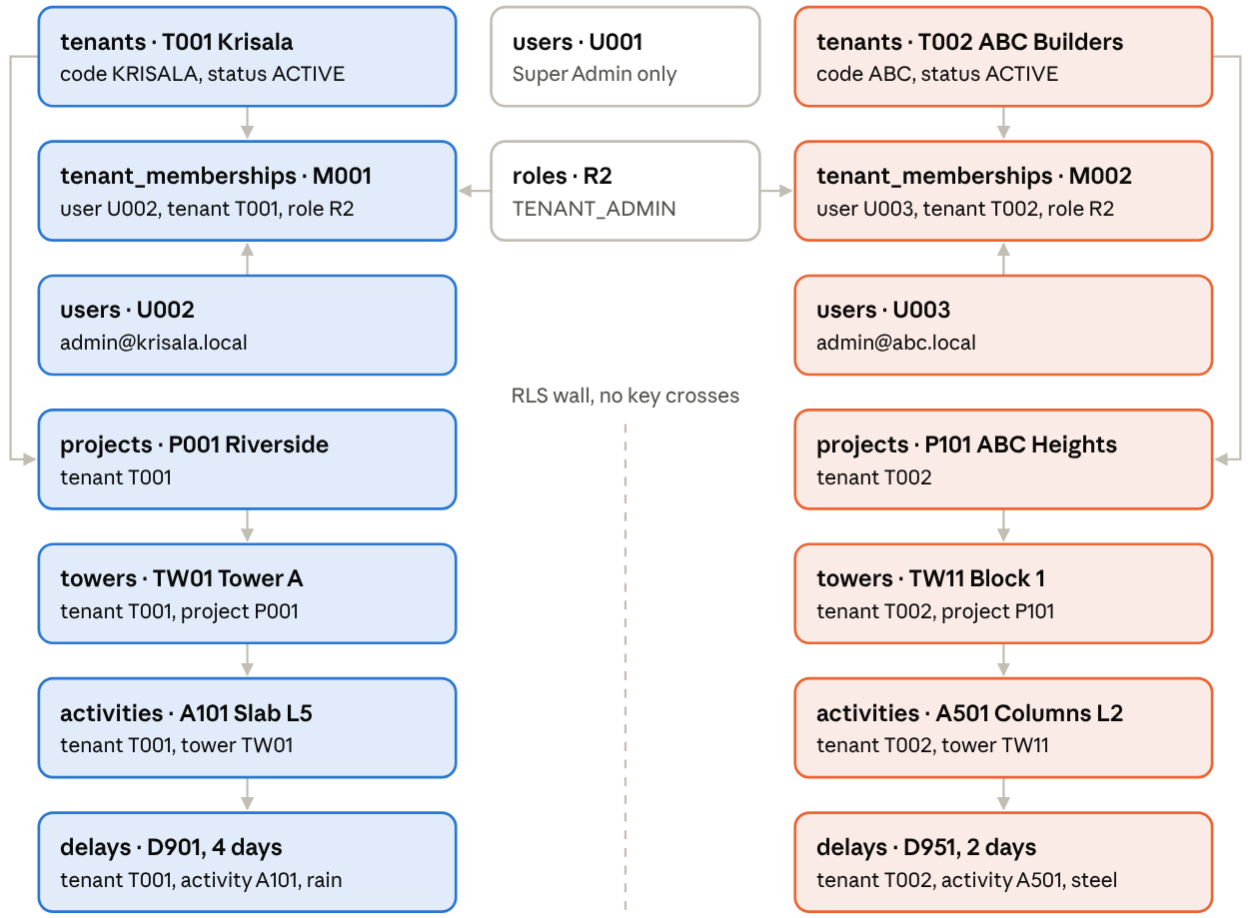
ALTER TABLE towers ENABLE ROW LEVEL SECURITY;
ALTER TABLE towers FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON towers
  USING      (tenant_id = current_setting('app.tenant_id', true)::uuid)
  WITH CHECK (tenant_id = current_setting('app.tenant_id', true)::uuid);
```

With that composite key, the database itself refuses a Krisala tower that points at an ABC project.

10.2 Example: Krisala and ABC Builders in one database

Krisala and ABC Builders are rows in the same tables; each row's foreign keys point only to rows of the same tenant. Readable IDs such as T001 stand in for UUIDs.

Krisala and ABC rows share tables but never share a key



Example rows for Krisala (T001) and ABC Builders (T002) in the same tables

Table	Krisala row	ABC Builders row
`tenants`	T001, Krisala, KRISALA, ACTIVE	T002, ABC Builders, ABC, ACTIVE
`users`	U002, admin@krisala.local	U003, admin@abc.local
`tenant_memberships`	M001: U002 → T001 as R2 TENANT_ADMIN	M002: U003 → T002 as R2 TENANT_ADMIN
`projects`	P001 Riverside, tenant T001	P101 ABC Heights, tenant T002
`towers`	TW01 Tower A, project P001	TW11 Block 1, project P101
`activities`	A101 Slab L5, tower TW01	A501 Columns L2, tower TW11
`delays`	D901: A101, 4 days, rain	D951: A501, 2 days, steel

`users` and `roles` are platform tables: U001 (Super Admin) has no membership, and role R2 is shared by both tenants' memberships.

How NestJS connects. One `pg` Pool logs in as `app_runtime` to the single database. Each request borrows a connection, opens a transaction, sets `app.tenant_id`, runs its queries and returns the connection clean. The same query text gives each tenant only its own rows:

```
-- inside withTenant(T001), no WHERE tenant_id needed
SELECT p.name, t.name AS tower, a.name AS activity, d.delay_days, d.cause
FROM delays d
JOIN activities a ON a.id = d.activity_id
JOIN towers t     ON t.id = a.tower_id
JOIN projects p   ON p.id = t.project_id;
-- Krisala gets: Riverside | Tower A | Slab L5 | 4 | rain
```

Statement	As Krisala (T001)	As ABC Builders (T002)
`SELECT * FROM delays`	D901 only	D951 only
`SELECT * FROM projects WHERE id = 'P101'`	0 rows	P101
`INSERT INTO towers (tenant_id, project_id, ...) VALUES ('T002', 'P101', ...)`	Rejected by RLS `WITH CHECK`	Allowed
`INSERT INTO towers ... VALUES ('T001', 'P101', ...)`	Rejected by the composite foreign key	Rejected by RLS `WITH CHECK`
`UPDATE delays SET delay_days = 0 WHERE id = 'D951'`	0 rows updated	1 row updated

The Super Admin's tenant list reads only the platform `tenants` table, so it shows both T001 and T002 without ever opening tenant data.

11. Repository and service structure

One monorepo with three deployable services; the Python service is added in Phase 3 and never imports the database package.

```
construction-intelligence-platform/
├── apps/
|   ├── web/                    # Next.js App Router
|   └── api/                    # NestJS
|       └── src/modules/
|           ├── auth/           # JWT, login, guards
```

```
|           ├── rbac/              # roles, decorators
|           ├── tenancy/           # membership, TenantDbService, withTenant()
|           ├── tenants/           # Super Admin tenant management
|           ├── tenant-config/     # branding, theme
|           ├── audit/
|           ├── construction/      # Phase 2: projects, activities, delays...
|           ├── ai-gateway/        # Phase 3: /ai/* for the UI, credit checks
|           └── agent-tools/       # Phase 3: /tools/* for the Python service
├── services/
|   └── ai/                        # Phase 3: Python + FastAPI
|       ├── app/agents/            # sun.py, earth.py, delay.py,
root_cause.py...
|       ├── app/tools/             # typed clients for NestJS /tools/*
|       ├── app/llm/               # provider abstraction, model routing
|       ├── app/ml/                # registry.py, features.py, training/
|       └── app/engines/           # rules, NetworkX CPM, SPI/CPI
├── packages/
|   ├── db/                        # Drizzle schema, migrations, RLS SQL
|   └── contracts/                 # OpenAPI/JSON schemas shared by api and ai
└── docs/architecture, docs/decisions
```

Service	Talks to	Never talks to
`web`	`api`	Database, `ai`
`api`	PostgreSQL, Redis, `ai`	LLM providers
`ai`	`api` `/tools/*`, LLM providers, model storage	PostgreSQL (except the separate training job)

The tool contracts in `packages/contracts` are the agreement between NestJS and Python. Each tool has an input schema, an output schema and a required scope, such as `schedule:read`.

12. Security rules

The tenant is always decided by NestJS from the authenticated membership; nothing downstream can choose a different one.

Layer	Rule
Browser	Never the source of `tenant_id`; any tenant value it sends is ignored or must match the membership

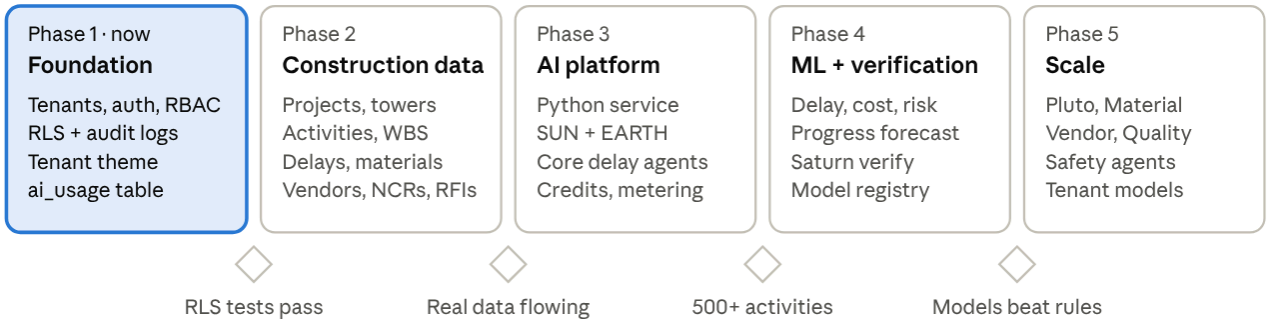
Layer	Rule
NestJS	`withTenant(tenantId, fn)` opens a transaction, runs `SET LOCAL app.tenant_id`, and passes the transaction to `fn`; no global tenant variable
PostgreSQL	RLS enabled and forced on tenant tables, `USING` and `WITH CHECK` policies, runtime role without `BYPASSRLS`
NestJS → Python	Short-lived signed internal token: tenant, user, request id, allowed scopes, expiry of a few minutes
Python → NestJS tools	Every tool call carries that token; NestJS re-checks tenant and scope, never trusts a tenant in the body
Agents	Read tools by default; write tools (for example raising an alert) are explicit, scoped and audited
LLM prompts	Data of one tenant per call; no tenant data stored by the provider beyond the call; prompts traced with `tenant_id`
Vectors	Every chunk has `tenant_id`; search runs through NestJS under RLS (or a server-derived Pinecone namespace)
Models	Tenant models loaded only for that tenant; base model trained only on consenting tenants
Logs and traces	`tenant_id` and `request_id` on every line

Required tests: tenant A cannot read or update tenant B through the API, through a query that omits the tenant filter, through an agent tool call with a tampered body, or through vector search.

13. Merged phase roadmap

The platform blueprint and the AI standard merge into five phases; only Phase 1 is built now, and each gate must pass before the next phase starts.

AI starts only after the tenant boundary and construction data exist



Merged roadmap · 5 phases, 4 gates

Gate	Pass criteria
1 → 2	Cross-tenant tests pass through the API and through queries without tenant filters; runtime role has no `BYPASSRLS`
2 → 3	At least one tenant is entering real projects, activities and delays daily
3 → 4	500+ historical activities with outcomes exist for training (the AI standard's minimum)
4 → 5	Each ML model beats the rules-only baseline on held-out data

Before building Phase 3, re-pin the Python libraries; the AI standard's `requirements.txt` lists old versions such as `langchain 0.1.0`.